Each generated text is added to the lag queue. From there, a batch the size of `example.countwords.delegation-parallelism` is sent to the router, which forwards the messages to the workers. Each request to a worker is executed by an `ask` with a timeout of 3 seconds. With four aggregator nodes, each with five workers, a delegation parallelism of 20 texts would be fully utilized.

When the worker's response comes back, the master sends itself `CountedWords` when the response is `Success` or `FailedJob` when the response is `Failure`.

Listing 8.10 Master's `working` method implementation, part 1

```
object Master {

  ...
  def working(
      workersRouter: ActorRef[Worker.Command],
      countedWords: Map[String, Int] = Map(),                    ❶
      lag: Vector[String] = Vector()): Behavior[Event] =         ❷
    Behaviors.setup[Event] { context =>

      implicit val timeout: Timeout = 3.seconds                  ❸
      val parallelism = ...                                      ❸
      Behaviors.receiveMessage[Event] {
        case Tick =>
          context.log.debug(s"receiving, current lag ${lag.size} ")

          val text = "this simulates a stream, a very simple stream"
          val allTexts = lag :+ text
          val (firstPart, secondPart) = allTexts.splitAt(parallelism) ❹
          firstPart.map { text =>
            context.ask(workersRouter, Worker.Process(text, _)) {    ❺
              case Success(CountedWords(map)) =>
                CountedWords(map)
              case Failure(ex) =>
                FailedJob(text)
            }
          }
          working(workersRouter, countedWords, secondPart)
    case ...
```